

ARROWARCH

One instruction in. Finished, proven work out. No coding knowledge needed to read this.

You say what you want. A crew of AI helpers works it out, does it, checks its own work, and hands it back only when it's actually proven. You step in twice, about a minute each.

Why every other AI coding tool disappoints you

You: "the reports show zero for insurance companies"
It: reads a bit, guesses a lot, writes code, says "Done!"
You: ...it isn't. And now three other things are broken.

Four reasons, every time. **The job is too big for one head** — an AI has a desk, and a whole project piled on it means papers fall off the back; it forgets the middle of its own work while still sounding confident. **"Done" just means it said so** — nobody checked. **It guesses what you meant**, so a deliberate choice gets "fixed" as a bug. **It starts from zero every session**, re-learning your project forever. ARROWARCH kills all four.

The cast

```
YOU  ---- "the Arrow": one clear thing you want
|
+-----+
| MISSION CONTROL | the front desk. Keeps the map of who's doing what.
|                 | Never plans, never builds. Checks everything.
+-----+
| THE PLANNER     | the brain. Cuts the job up. Can call in MORE
|                 | Planners for the hard pieces.
+-----+
| THE BUILDERS    | the hands. One small job each, own private copy
|                 | of the project, gone the moment they finish.
+-----+

Always alongside: THE LIBRARY - answers we already paid to learn.
                  THE LEDGER  - the receipt book. No receipt, no "done".
```

One job, start to finish

```
YOU      "reports show zero for insurance companies"
MISSION  small, medium or big? -> big. Wake the Planner.
CONTROL
PLANNER  goes and LOOKS first – the real code, the real reports, the real
          files. Finds the real cause. Spots something odd -> does NOT
          decide -> writes it down as a question for you.

GATE 1   "Here's what I found. Here's the one thing only you can rule on."
YOU      "that one's on purpose, leave it."          <- 1 minute
PLANNER  cuts the job into 2 chunks, each into small jobs. Writes every
          Builder a one-page brief: what to change, which files it may
          touch, and how we'll know it worked.

GATE 2   a 20-line summary: what we're building, and what we are NOT.
YOU      "go."          <- 1 minute
BUILDERS two start at once, each in a private copy, so they can't tread
          on each other.

INSPECTOR Mission Control re-runs every test ITSELF. It does not read the
          Builder's summary and believe it.

MERGE    one at a time, each re-tested against everyone else's finished
          work before it's allowed in.

DONE     Your total involvement: two short reads, two words.
```

The new part: Planners calling Planners

A Planner holding the whole job fills up and starts forgetting. So it doesn't. When a piece is too hard, it **calls in another Planner for just that piece** — and so can that one.

```
PLANNER #1 sees: the whole Arrow -> splits into "reports" + "the screen" -> leaves
|
+-- PLANNER #2 sees: ONLY "reports" -> splits into 3 small jobs -> leaves
|      +-- starts its own Builders
|      +-- hands out the private copies
|      +-- collects and answers for the results
|
+-- PLANNER #3 sees: ONLY "the screen" – one job is still too hard
      +-- PLANNER #4 sees: only THAT job
```

Each Planner sees a smaller slice than its parent, so **nobody ever holds the whole job**. A Planner isn't an advisor — it runs its own branch end to end and answers for it.

How nobody forgets

EVERYONE ELSE	ARROWARCH
one desk, whole project	small desk, one job
+-----+	+-----+ +-----+ +-----+
everything, always	job A job B job C
... falling off ..	+-----+ +-----+ +-----+
+-----+	\ /
	v v v
forgets the middle	+-----+
	THE MAP (on disk)
	the only real truth
	+-----+

The Map is a plain file on the hard drive — outside every AI's head. Any helper can crash, fill up or be switched off mid-sentence and **nothing is lost**; a fresh one picks up exactly where the old one stopped. It's also how the re-learning problem dies: one helper studies your project properly, once, and writes it onto The Map.

The Sizing Question — our sharpest edge

Every AI builder has one of two diseases: it builds a cathedral when you asked for a shed, or it skips the roof. So when a helper gets **stuck on a decision**, it stops and runs this ladder out loud, in order:

```
STUCK
1 Does this need to exist at all?  -> no  -> delete it. Done. Cheapest win there is.
2 Do we already have it?          -> yes -> use that. Never write a second one.
3 Does the toolkit already do it? -> yes -> use that.
4 Can it be ONE small thing?      -> yes -> build exactly that, nothing more.
5 Still genuinely unsure?         -> ask YOU. One plain question, one example.
```

T00 MUCH	JUST RIGHT	T00 LITTLE
building for a	building for the	skipping the
customer who does	one person actually	safety rails to
not exist yet	in front of you	look fast

Five things are never "too much", and cutting them isn't lean — it's broken: checking what the user typed, handling things going wrong, locking the doors, making it usable by people with disabilities, and tests. And no helper may invent a setting, a switch or a "flexible layer" for a second user who doesn't exist yet. Build the one real case.

The Library — the shelf we check before guessing

A growing collection of answers we already paid for in time: which tools are genuinely free and to what limit, which service quietly shut down, and a log of **problems we already solved and what actually fixed them**.

```
helper hits a problem
+-- 1. check THE LIBRARY -> known problem? use the known fix. Zero cost.
+-- 2. not there?        -> work it out, then WRITE IT BACK to the shelf.
```

Every mistake gets paid for exactly once, ever — by anyone, on any project, forever.

The Ledger — nothing counts because someone said so

Builder: "done, tests pass" ← an opinion. Worth nothing.

INSPECTOR then:

- 1 runs every test again itself → the result decides, not the summary
- 2 checks which files ACTUALLY changed → touched one it wasn't allowed? stop.
- 3 checks no existing test was made easier → the oldest trick in the book
- 4 compares to what was broken BEFORE it → judged only on what IT broke

Pass → a dated receipt goes in The Ledger. Later steps CITE that receipt instead of re-running everything. A receipt older than the code it describes is STALE and void.

Failures are sorted by REASON, because retrying only helps one kind:

- wrong code → retry, worth a try needs a locked file → back to the Planner
- wrong password → NEVER retry, come to you the plan is wrong → back to Gate 2

A job may only start when all five are true: its chunk is open · what it waits on has landed · you approved the plan · nothing it needs is awaiting your ruling · no other running job touches the same files. That last one is worked out by plain arithmetic, never by an AI's opinion. Judgement sits *on top of* counted facts, never instead of them.

What we took from gstack

Their idea	What it becomes here
Receipts for tests, marked fresh or stale	The Ledger — the backbone, not an add-on
Locking which files may be touched	Every brief names its files; anything else stops the run
Auto-saving work as it goes	A crash costs nothing; the next helper resumes
A second, independent reviewer	Risky work gets one extra read. One opinion, not a loop
Forcing questions before building	Folded into The Sizing Question
Shared memory across projects	The Library

Why this is different

- **Nobody holds the whole job.** Planners call Planners, each seeing less than its parent.
- **The truth lives on disk, not in an AI's head.** A helper can die mid-sentence; nothing is lost.
- **"Done" is proven, never claimed.** The Inspector re-does the checking itself, every time.
- **Being stuck triggers a real question** — does this even need to exist? — not a confident guess.
- **Mistakes are paid for once, forever,** because The Library remembers them.
- **You spend two minutes,** at the two moments where only a human can decide.